Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Национальный исследовательский университет «МЭИ»

Кафедра управления и интеллектуальных технологий (УИТ)

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 1

по дисциплине «Управление большими системами»

Тема: «Структурный анализ больших систем управления. Упорядочение информационной структуры и оценка времени подготовки документов»
(Вариант № 3)

Выполнили студенты гр. А-03-23:
Грудинин Е.К.
Турханов А.К.

Проверил:
ассистент Обычайко Д.С.

Москва, 2026 г.

Цель работы

Приобретение практических навыков формализованного описания и анализа потоков информации в больших системах управления, включая алгоритмизацию построения порядковой функции на ориентированном графе без контуров, выделение иерархических уровней информационной структуры, приведение матрицы смежности к верхнетреугольному виду и расчет минимальных путей прохождения документов методом динамического программирования.

1. Матрица смежности графа с индивидуально дополненными дугами

Информационная структура документооборота представлена ориентированным графом $G = (V, E)$, вершины которого соответствуют документам, а дуги отражают отношения вхождения и порядка. Базовый граф включает 17 вершин и 22 дуги с заданными ценами (временем подготовки документов). В соответствии с Вариантом № 3 граф дополнен двумя дугами (таблица вариантов в методических указаниях):

1. Дуга (11, 4) с весом $q(11,4) = 5$;
2. Дуга (6, 17) с весом $q(6,17) = 5$.

Итоговый граф Варианта 3 содержит 17 вершин и 24 направленные дуги. Матрица смежности и цен дуг исходного графа представлена в таблице 1. Элемент a_{ij} равен весу дуги $q(a_i, a_j)$, а при отсутствии связи равен нулю. Дополнительные дуги Варианта 3 отмечены в таблице цветовым фоном.

Таблица 1 – Матрица смежности и цен исходного графа (Вариант 3)

i \ j	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
1	0								2								
2		0															2
3			0		4		5				3						
4				0					2								
5				3	0	4											
6						0						3					5
7	2			2			0										
8								0						2			
9									0							3	
10										0					1		
11	3	4		5							0						
12												0				2	
13													0		2		
14														0	2		
15															0		
16										3			3			0	
17								2									0

2. Исходное графическое представление системы

На основе матрицы смежности построено графическое представление исходного графа системы до упорядочения (рисунок 1). Входным полюсом (узлом генерации первичной информации) выступает вершина 3, в которую не входит ни одна дуга. Выходным полюсом (узлом завершения документооборота) является вершина 15, не имеющая исходящих связей. Хаотичный характер пересечений дуг иллюстрирует сложность восприятия многоуровневой структуры без предварительного упорядочения.

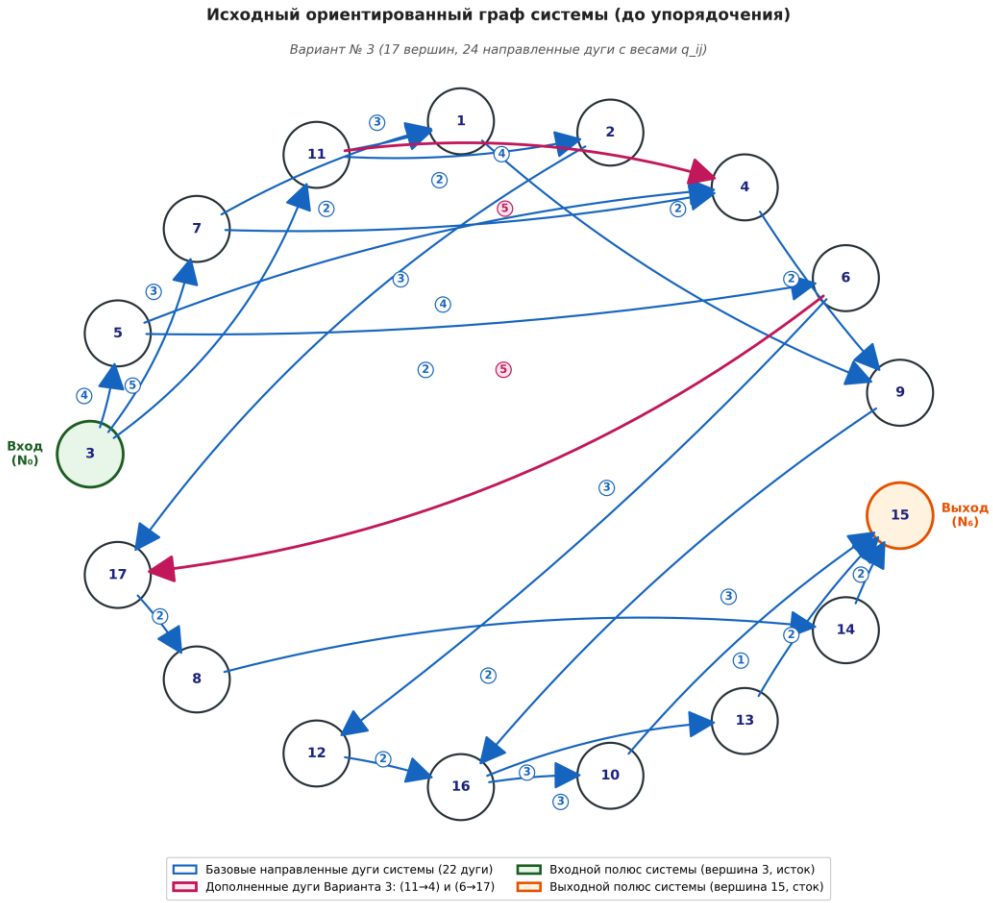


Рисунок 1 – Исходный ориентированный граф системы (до упорядочения) для Варианта 3

3. Графическое представление системы, упорядоченной по уровням, с отображением минимального пути

3.1. Алгоритм введения порядковой функции

Введение порядковой функции на ориентированном графе без контуров разбивает множество вершин на непересекающиеся подмножества (уровни) $N_0, N_1, \dots, N_m$. Вершины нулевого уровня N_0 определяются отсутствием входящих связей (пустое множество левых инциденций $G^{-1}(i) = \emptyset$):

$$N_0 = \{i \in V \mid G^{-1}(i) = \emptyset\}$$

Каждый последующий уровень N_m формируется из еще не распределенных вершин, все входящие дуги в которые исходят исключительно из вершин ранее сформированных уровней:

$$N_m = \{i \in V \setminus (N_0 \cup \dots \cup N_{m-1}) \mid G^{-1}(i) \subseteq (N_0 \cup \dots \cup N_{m-1})\}$$

3.2. Результаты упорядочения графа Варианта 3

Пошаговое применение алгоритма позволило разделить граф Варианта 3 на 7 иерархических уровней:

• Уровень N_0: вершины {3} (G^{-1}(3) = ∅ (исходный документ / входной полюс));

• Уровень N_1: вершины {5, 7, 11} (дуги входят только из N_0: G^{-1}(5)={3}, G^{-1}(7)={3}, G^{-1}(11)={3});

• Уровень N_2: вершины {1, 2, 4, 6} (дуги входят только из N_0 ∪ N_1);

• Уровень N_3: вершины {9, 12, 17} (дуги входят только из N_0 ∪ N_1 ∪ N_2);

• Уровень N_4: вершины {8, 16} (дуги входят только из N_0 ∪ ... ∪ N_3);

• Уровень N_5: вершины {10, 13, 14} (дуги входят только из N_0 ∪ ... ∪ N_4);

• Уровень N_6: вершины {15} (дуги входят только из N_0 ∪ ... ∪ N_5 (итоговый документ / сток));

После формирования уровней вершины получают новую сквозную нумерацию от 1 до 17 (таблица 2). В новой нумерации матрица смежности приводится к строго верхнетреугольному виду:

$$a_{ij} = 0 \text{при} i \geq j$$

Таблица 2 – Соответствие исходной и новой топологической нумерации вершин

Уровень:	N_0	N_1	N_1	N_1	N_2	N_2	N_2	N_2	N_3	N_3	N_3	N_4	N_4	N_5	N_5	N_5	N_6
Новый №:	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**	**17**
Исходный №:	3	5	7	11	1	2	4	6	9	12	17	8	16	10	13	14	15

Таблица 3 – Упорядоченная верхнетреугольная матрица цен смежности (Вариант 3)

Нов.(исх.)	#1	#2	#3	#4	#5	#6	#7	#8	#9	#10	#11	#12	#13	#14	#15	#16	#17
#1 (3)	0	4	5	3													
#2 (5)		0					3	4									
#3 (7)			0		2		2										
#4 (11)				0	3	4	5										
#5 (1)					0				2								
#6 (2)						0					2						
#7 (4)							0		2								
#8 (6)								0		3	5						
#9 (9)									0				3				
#10 (12)										0			2				
#11 (17)											0	2					
#12 (8)												0				2	
#13 (16)													0	3	3		
#14 (10)														0			1
#15 (13)															0		2
#16 (14)																0	2
#17 (15)																	0

В полученной упорядоченной матрице все ненулевые элементы расположены строго выше главной диагонали ($a_{ij} = 0 \text{при} i \geq j$), что подтверждает отсутствие контуров и корректность топологического упорядочения системы.

3.3. Расчет минимального пути методом динамического программирования

Поиск минимального пути на графе без контуров реализуется на основе функционального уравнения Беллмана:

$$q_s^{min}(a_1, a_j) = min_{a_i \in G^{-1}(a_j)}[q_s^{min}(a_1, a_i) + q(a_i, a_j)]$$

Здесь величина $q_s^{min}(a_1, a_j)$ обозначает минимальную суммарную оценку пути из источника a_1 в вершину a_j, а величина $q(a_i, a_j)$ задает вес соединяющей их дуги. За счет треугольного вида матрицы вычисления выполняются за один прямой проход по упорядоченным вершинам:

• Вершина #1 (исх. 3): источник, F(3) = 0;

• Вершина #2 (исх. 5): min(q(3,5)+F(3)=4+0=4) = 4;

• Вершина #3 (исх. 7): min(q(3,7)+F(3)=5+0=5) = 5;

• Вершина #4 (исх. 11): min(q(3,11)+F(3)=3+0=3) = 3;

• Вершина #5 (исх. 1): min(q(7,1)+F(7)=2+5=7, q(11,1)+F(11)=3+3=6) = 6;

• Вершина #6 (исх. 2): min(q(11,2)+F(11)=4+3=7) = 7;

• Вершина #7 (исх. 4): min(q(5,4)+F(5)=3+4=7, q(7,4)+F(7)=2+5=7, q(11,4)+F(11)=5+3=8) = 7;

• Вершина #8 (исх. 6): min(q(5,6)+F(5)=4+4=8) = 8;

• Вершина #9 (исх. 9): min(q(1,9)+F(1)=2+6=8, q(4,9)+F(4)=2+7=9) = 8;

• Вершина #10 (исх. 12): min(q(6,12)+F(6)=3+8=11) = 11;

• Вершина #11 (исх. 17): min(q(2,17)+F(2)=2+7=9, q(6,17)+F(6)=5+8=13) = 9;

• Вершина #12 (исх. 8): min(q(17,8)+F(17)=2+9=11) = 11;

• Вершина #13 (исх. 16): min(q(9,16)+F(9)=3+8=11, q(12,16)+F(12)=2+11=13) = 11;

• Вершина #14 (исх. 10): min(q(16,10)+F(16)=3+11=14) = 14;

• Вершина #15 (исх. 13): min(q(16,13)+F(16)=3+11=14) = 14;

• Вершина #16 (исх. 14): min(q(8,14)+F(8)=2+11=13) = 13;

• Вершина #17 (исх. 15): min(q(10,15)+F(10)=1+14=15, q(13,15)+F(13)=2+14=16, q(14,15)+F(14)=2+13=15) = 15;

Итоговое минимальное время прохождения документов составляет $L_{min} = 15$. В исследуемой системе присутствуют два равноценных оптимальных маршрута:

Маршрут 1:

• В исходной нумерации: 3 -> 11 -> 1 -> 9 -> 16 -> 10 -> 15

• В новой нумерации: #1 -> #4 -> #5 -> #9 -> #13 -> #14 -> #17

• Сумма весов дуг: 3 + 3 + 2 + 3 + 3 + 1 = 15.

Маршрут 2:

• В исходной нумерации: 3 -> 11 -> 2 -> 17 -> 8 -> 14 -> 15

• В новой нумерации: #1 -> #4 -> #6 -> #11 -> #12 -> #16 -> #17

• Сумма весов дуг: 3 + 4 + 2 + 2 + 2 + 2 = 15.

На рисунке 2 представлена схема упорядоченной системы (точный электронный аналог рукописного расчета) с выделением найденных минимальных путей.

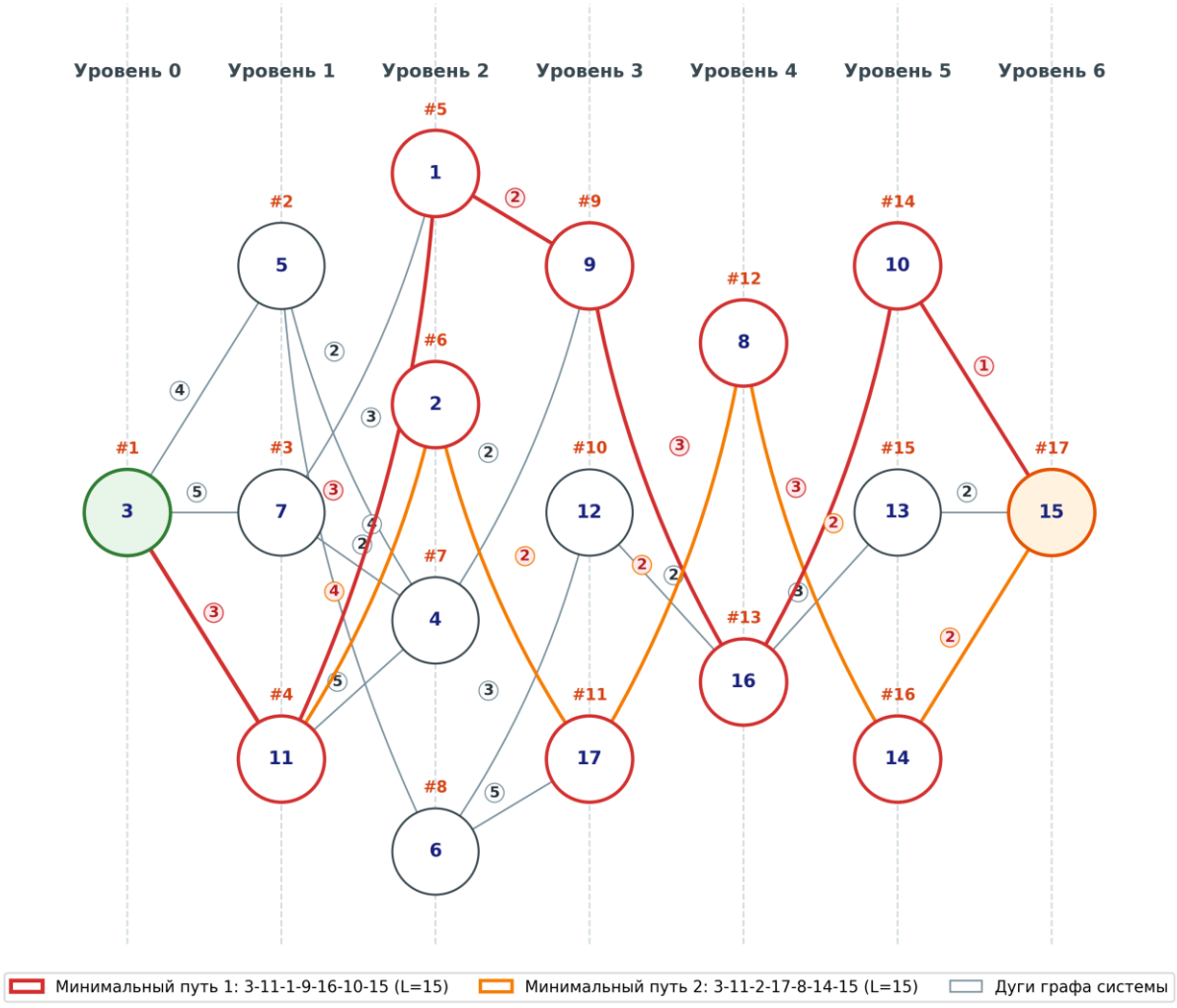


Рисунок 2 – Графическое представление системы, упорядоченной по уровням, с отображением минимальных путей

4. Блок-схемы алгоритмов решения задач

Разработаны блок-схемы алгоритмов решения задач: алгоритма упорядочения вершин по уровням (рисунок 3) и алгоритма определения минимального пути методом динамического программирования (рисунок 4).

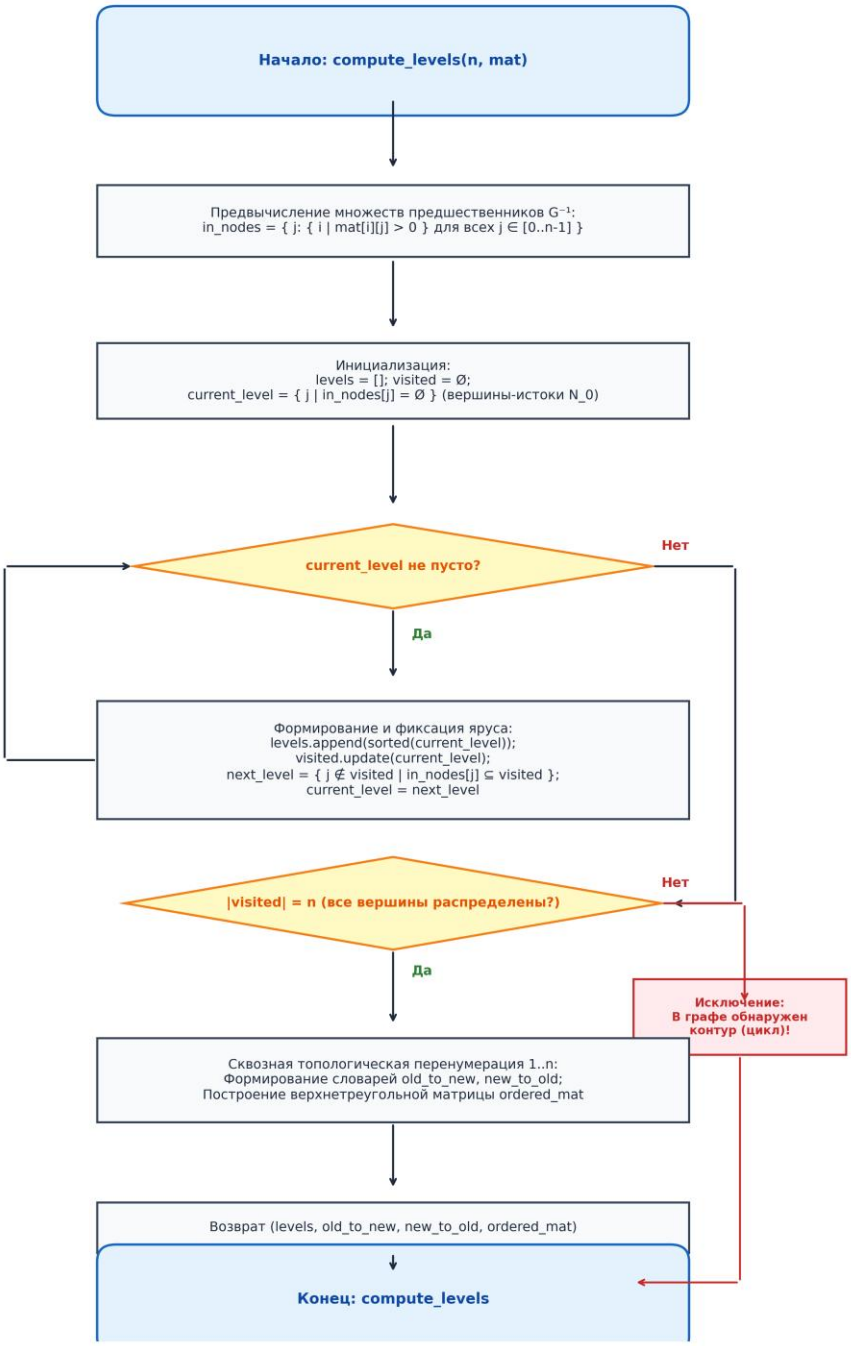


Рисунок 3 – Блок-схема алгоритма упорядочения вершин ориентированного графа по уровням

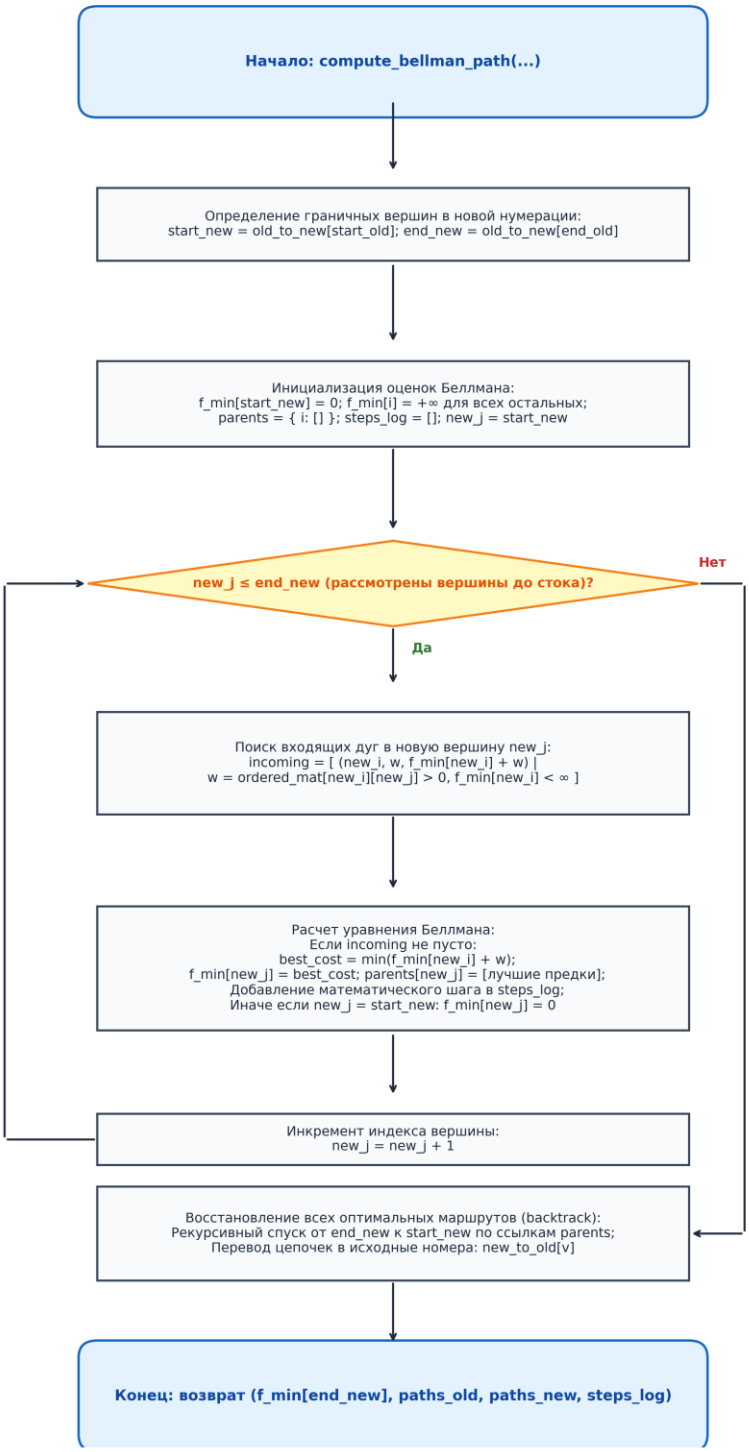


Рисунок 4 – Блок-схема алгоритма определения минимального пути методом динамического программирования

5. Листинг разработанной программы

Ниже представлен исходный текст ключевых расчетных функций программы: алгоритма поуровневого упорядочения вершин графа с приведением матрицы к верхнетреугольному виду и алгоритма расчета минимального пути методом динамического программирования (уравнение Беллмана):

```
def compute_levels(n, mat):
    """
    Поуровневое разбиение вершин графа без контуров (DAG).
    Возвращает:
      - levels: список уровней со списками вершин (нумерация 1..n);
      - old_to_new: словарь {исходный_номер: новый_номер};
      - new_to_old: словарь {новый_номер: исходный_номер};
      - ordered_mat: верхнетреугольная матрица цен в новой нумерации.
    """
    # Множество непосредственных предшественников для каждой вершины
    in_nodes = {j: {i for i in range(n) if mat[i][j] > 0} for j in range(n)}

    levels = []
    visited = set()
    current_level = {j for j, preds in in_nodes.items() if not preds}

    while current_level:
        levels.append(sorted(current_level))
        visited.update(current_level)

        next_level = {
            j for j in range(n)
            if j not in visited and in_nodes[j].issubset(visited)
        }
        current_level = next_level

    if len(visited) != n:
        missing = sorted(v + 1 for v in set(range(n)) - visited)
        raise ValueError(f"В графе обнаружен контур (цикл)! Невозможно упорядочить вершины: {missing}")

    levels_1based = [[v + 1 for v in lvl] for lvl in levels]

    old_to_new = {}
    new_to_old = {}
    new_num = 1
    for lvl in levels_1based:
        for old_v in lvl:
            old_to_new[old_v] = new_num
            new_to_old[new_num] = old_v
            new_num += 1

    ordered_mat = [[0] * n for _ in range(n)]
    for i in range(1, n + 1):
        for j in range(1, n + 1):
            ordered_mat[i - 1][j - 1] = mat[new_to_old[i] - 1][new_to_old[j] - 1]

    return levels_1based, old_to_new, new_to_old, ordered_mat


def compute_bellman_path(n, ordered_mat, new_to_old, old_to_new, start_old=3, end_old=15):
    """
    Поиск минимального пути методом динамического программирования (уравнение Беллмана).
    Возвращает:
      - min_cost: минимальная длина пути (или None, если пути нет);
      - paths_old: список оптимальных маршрутов в исходной нумерации;
      - paths_new: список оптимальных маршрутов в новой нумерации;
      - steps_log: пошаговый протокол расчета уравнений Беллмана.
    """
```

```
    start_new = old_to_new.get(start_old, 1)
    end_new = old_to_new.get(end_old, n)

    f_min = {i: float('inf') for i in range(1, n + 1)}
    parents = {i: [] for i in range(1, n + 1)}
    f_min[start_new] = 0

    steps_log = []

    # Движение в топологическом порядке новых номеров (1 .. n)
    for new_j in range(start_new, end_new + 1):
        old_j = new_to_old[new_j]
        incoming = []

        for new_i in range(start_new, new_j):
            w = ordered_mat[new_i - 1][new_j - 1]
            prev_cost = f_min[new_i]
            if w > 0 and prev_cost != float('inf'):
                incoming.append((new_i, new_to_old[new_i], w, prev_cost + w))

        if incoming:
            best_cost = min(item[3] for item in incoming)
            f_min[new_j] = best_cost
            parents[new_j] = [item[0] for item in incoming if item[3] == best_cost]

            variants = [
                f"q({old_i},{old_j})+F({old_i})={w}+{f_min[new_i]}={cand}"
                for new_i, old_i, w, cand in incoming
            ]
            steps_log.append(f"Вершина #{new_j} (исх. {old_j}): min({', '.join(variants)}) =
{best_cost}")
        elif new_j == start_new:
            steps_log.append(f"Вершина #{new_j} (исх. {old_j}): источник, F({old_j}) = 0")

    if f_min[end_new] == float('inf'):
        return None, [], [], steps_log

    # Восстановление всех оптимальных маршрутов (backtracking)
    paths_new = []

    def backtrack(curr, current_path):
        if curr == start_new:
            paths_new.append(list(reversed(current_path)))
            return
        for p in parents[curr]:
            backtrack(p, current_path + [p])

    backtrack(end_new, [end_new])
    paths_old = [[new_to_old[v] for v in p] for p in paths_new]

    return f_min[end_new], paths_old, paths_new, steps_log
```

6. Результаты работы программы

Результаты решения задач структурного анализа, поуровневого упорядочения графа и определения минимального пути прохождения документов на экранных формах разработанной программы представлены на рисунках 5–7.

На рисунке 5 приведена экранная форма с введенной матрицей смежности и цен графа для Варианта 3 (17 вершин и 24 направленные дуги).

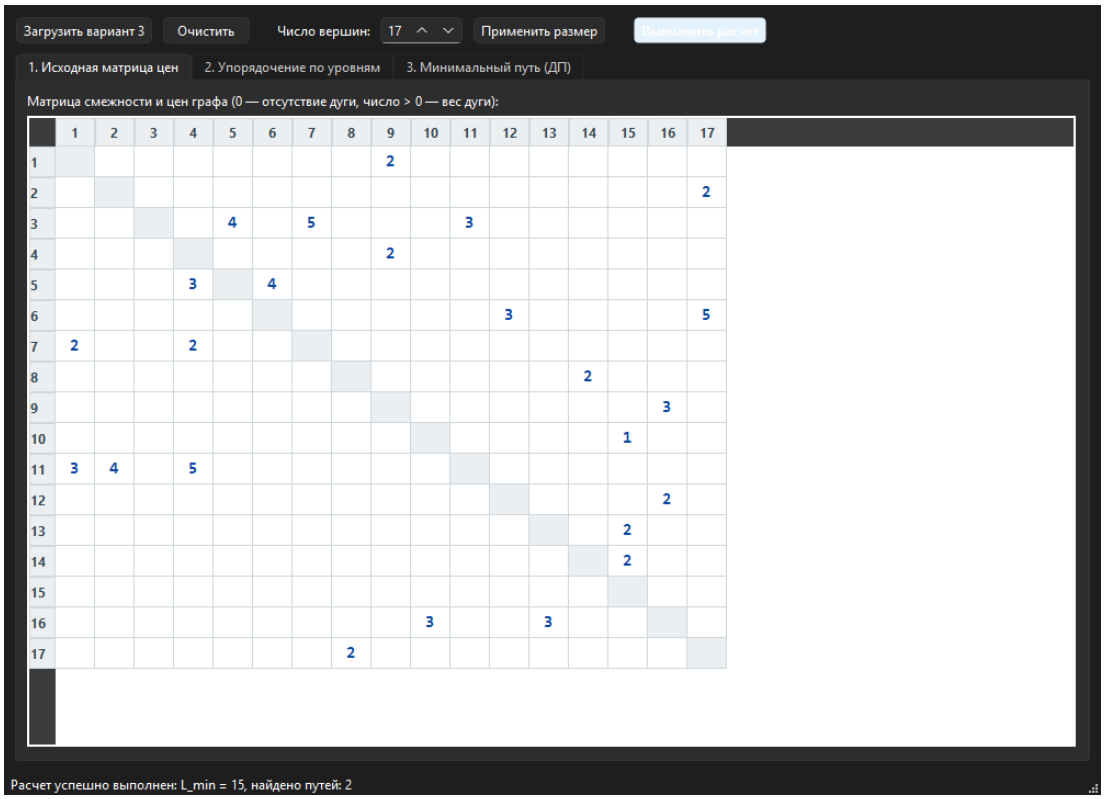


Рисунок 5 – Экранная форма с загруженной матрицей смежности и цен графа (Вариант 3)

На рисунке 6 представлены результаты разбиения вершин графа на 7 иерархических уровней ($N_0 \dots N_6$) и упорядоченная матрица цен. Расположение всех ненулевых элементов строго выше главной диагонали подтверждает ацикличность графа и корректность топологической перенумерации.

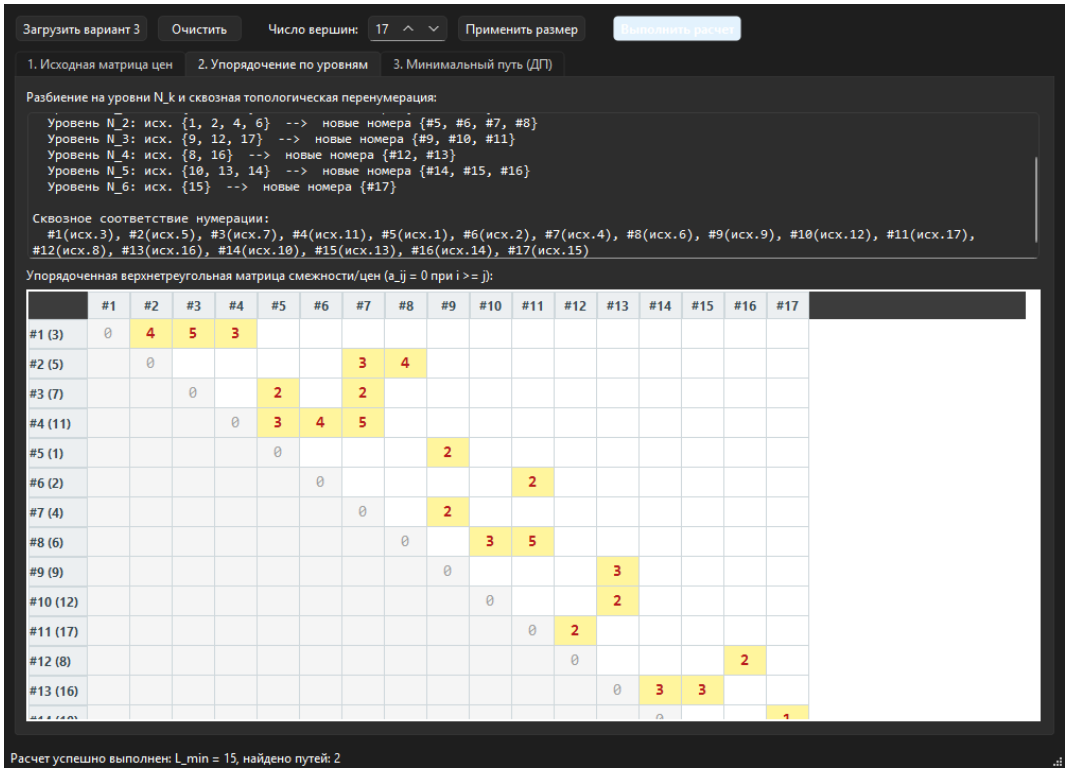


Рисунок 6 – Экранная форма распределения вершин по уровням и упорядоченной матрицы

На рисунке 7 представлены результаты пошагового расчета минимального пути по уравнениям Беллмана между источником 3 и стоком 15. Программа вывела подробный протокол расчета, минимальную длину пути $L_{min} = 15$ и оба альтернативных оптимальных маршрута.

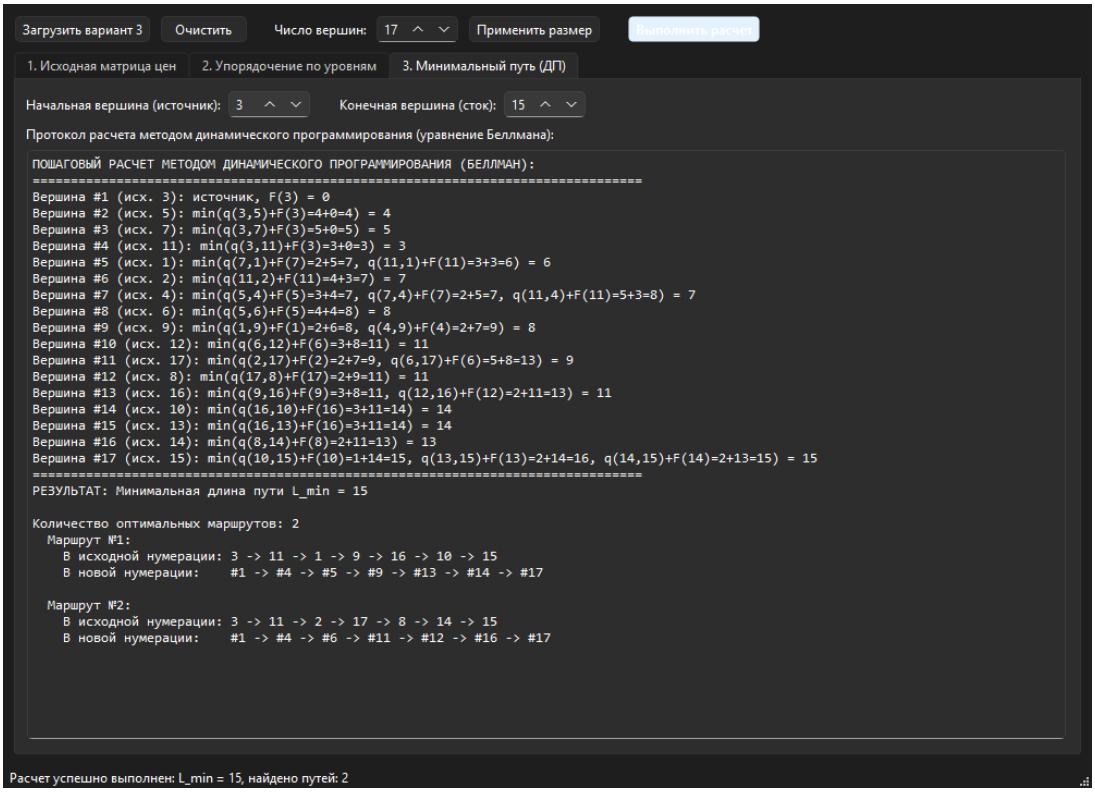


Рисунок 7 – Экранная форма пошагового расчета минимального пути по уравнениям Беллмана

Заключение

В результате выполнения лабораторной работы были получены следующие результаты:

• Для Варианта № 3 сформирована полная матрица смежности и цен графа документооборота из 17 вершин и 24 дуг.

• С помощью алгоритма построения порядковой функции выполнено разбиение графа на 7 иерархических уровней (N_0 ... N_6) и произведена топологическая перенумерация, обеспечившая строго верхнетреугольный вид матрицы цен.

• Методом динамического программирования (уравнение Беллмана) определено минимальное время прохождения документов L_min = 15 и найдены два равноценных альтернативных маршрута:

1) 3 -> 11 -> 1 -> 9 -> 16 -> 10 -> 15 (время: 3 + 3 + 2 + 3 + 3 + 1 = 15);
2) 3 -> 11 -> 2 -> 17 -> 8 -> 14 -> 15 (время: 3 + 4 + 2 + 2 + 2 + 2 = 15).

• Разработано программное обеспечение на языке Python, реализующее алгоритм упорядочения по уровням, контроль графа на контуры и пошаговый расчет минимального пути с выводом подробного протокола.